Lab Exercise: jQuery AJAX

Objective: Learn how to interact with the JSONPlaceholder API using jQuery AJAX to perform basic operations (GET, POST, PUT, DELETE) and manipulate data in the DOM. This lab will cover basic AJAX operations, logging responses, parsing JSON, and displaying data dynamically.

Initial Setup

1. Run the followingHTML code using Live Server. Gradually enhance it by completing the tasks below.

2. Test your code in the browser and observe how the DOM changes dynamically with each AJAX request.

3. Each task builds on the previous one. Follow the steps carefully.

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <title>Simple jQuery AJAX Lab Activity</title>

    <style>

        body { font-family: Arial; margin: 20px; }

        button { margin: 5px; padding: 10px; }

        #output { margin-top: 20px; white-space: pre-wrap; background: #f9f9f9; padding: 10px;
border: 1px solid #ccc; }

        #userList { list-style-type: none; padding: 0; }

        #userList li { padding: 10px; background-color: #f1f1f1; margin: 5px 0; border-radius:
5px; }

    </style>

</head>

<body>

    <h1>Simple jQuery AJAX Lab Activity</h1>



    <button id="btnGetAll">GET All Users</button><br>

    <div id="output">Results will appear here...</div>

    <ul id="userList"></ul>



    <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>

    <script>

       $(document).ready(function () {
```

```
    $('#btn1').click(function(){

        //get all users from backend-jsonplaceholder.com

        console.log('1');

        $.ajax({

            url:'https://dummyjson.com/products',

            success: function(result){

                console.table(result);

                console.log('2');

            },

            error: function(error){

                console.log(error);

                console.log('3');

            }

        });

        console.log('4');

    });


})


    </script>

</body>

</html>
```

.

Task 1: Make a GET Request and Log the Status

Goal: Learn how to make a GET request and handle success and error responses.

What you need to do:
1. Call the GET request to fetch users from https://jsonplaceholder.typicode.com/users.
2. On success, log the status (e.g., 'Request succeeded') to the console.
3. On error, log the error message to the console.

JavaScript code:

_____

_____

Task 2: Parse JSON Response and Compare String vs JSON Object

Goal: Understand how to handle the JSON response and compare string vs object formats.

What you need to do:
1. Fetch data using the GET request (https://jsonplaceholder.typicode.com/users).
2. On success, log the responseText (which will be a string) to the console.
3. Use JSON.parse() to convert the string to a JSON object.
4. Compare the string and the object in the console by logging them separately.
5. Use console.table() to display the JSON object in a readable tabular format.

JavaScript code:

_____

_____

Task 3: Display Users Using <ul> and

Goal: Learn to manipulate the DOM by dynamically displaying data using ul and li.

What you need to do:
1. On success, iterate through the user data (fetched from the API) and create a <ul> list.
2. For each user, create an <li> element and append it to the list.
3. The <li> should display the user name, email, and phone.
4. Insert the list into the DOM.

JavaScript code:

_____

_____

Task 4: Update a User Using PUT Request

Goal: Learn how to update a user's data using a PUT request.

What you need to do:
1. Use the PUT method in $.ajax() to update a user's details.
2. Pre-fill the form with new values for a specific user (you can use User ID to select which user to update).
3. When you update a user's data (e.g., name, email, phone), display the updated information on the page.

JavaScript code:

_____

_____

Task 5: Delete a User Using DELETE Request

Goal: Learn how to delete a user from the database using a DELETE request.

What you need to do:
1. Use the DELETE method in $.ajax() to remove a user by ID.
2. After the delete request, display a success message such as 'User deleted successfully'.
3. Refresh the users list by clicking the GET All Users button after a successful deletion.

JavaScript code:

_____

_____

Task 6: Create a New User Using POST Request

Goal: Learn how to create a new user by sending a POST request.

What you need to do:
1. Create a form for adding a new user.
2. On form submission, send a POST request with the user data to https://jsonplaceholder.typicode.com/users.
3. After the user is created, display their details on the page.

JavaScript code:

_____

_____

Task 7: Search Users by Name

Goal: Filter the list of users based on the name entered by the user in the search box.

What you need to do:
1. Implement a search box that filters users by their name.
2. As the user types, use jQuery's .on('input') event to dynamically search the users by name.
3. Only display the users whose names contain the search query.

JavaScript code:

_____

_____